API & System Integration Handbook

Department Charter, Corporate Workflow, Onboarding, 90-Day Roadmap and First Practical Assignment: from Beginner to Supervised Commercial Delivery

Developed by **V. LOKESHWARAN**
VEL Software Solutions
velsoftwaresolutions.in@gmail.com

Document VEL-API-001 v1.0 · Module 1 · Draft for management approval · November 2026

Contents

1. VEL API & System Integration Handbook
2. Part 1: The Department: Mission, Services and Boundaries
3. Part 2: Roles, Responsibilities and Career Path
4. Part 3: The Corporate Workflow: 16 Stages from Lead to Maintenance
5. Part 4: API Governance and Company Standards
6. Part 5: Beginner Onboarding Programme
7. Part 6: The 90-Day Learning Roadmap
8. Part 7: The Nine-Level Curriculum
9. Part 8: First Practical Assignment: The Client Registry API
10. Part 9: Internal Portal, Commercial Pricing and Client Acquisition
11. Appendices: Templates and Reference

All prices and market figures in this handbook's examples are placeholders until verified from official sources (see Stage 06). Structural, legal and tax matters need qualified review.

VEL API & System Integration Handbook

API Engineering and Enterprise System Integration Department: from Beginner to Supervised Commercial Delivery

Document control	
Document ID	VEL-API-001
Version	1.0 (draft for management approval)
Owner	V. LOKESHWARAN, Founder
Audience	New engineers, trainees, analysts and support staff with basic computer knowledge
Companion document	VEL Delivery Handbook (VEL-DH-001), the company-wide project procedure
Review cycle	Every quarter, after any production incident, and after any failed project
Contact	velsoftwaresolutions.in@gmail.com

0.1 What this handbook is

This is the single reference for the **API & System Integration Department** of VEL Software Solutions. It does three jobs:

1. **Defines the department**: what we sell, who does what, and the standards every API and integration must meet.
2. **Defines the delivery workflow**: sixteen stages that take a client from first enquiry to a monitored, maintained, handed-over integration.
3. **Trains the employee**: a 90-day programme that takes a beginner to a person who can join a *supervised* commercial project.

It is written in plain words. You do not need to know everything before you start. You need to **follow the stages in order, finish every checklist and collect every sign-off**.

> **This is Module 1 of a progressive series.** This volume gives you the handbook, the corporate workflow, onboarding, the 90-day roadmap, responsibilities and your first practical API assignment. Later modules (one per curriculum level) are released one at a time as the trainee completes each gate. See section 8.17 for the module release plan.

0.2 How to use it

1. Find your current week in the **90-day roadmap** (Part 6).
2. Read that week's lesson material and the linked official source **before** you write code.
3. Do the numbered steps in order. Tick the **Done checklist** as you go.
4. Save every deliverable in the exact folder named in the task.
5. Ask for the **gate sign-off**. You may not start the next phase without it.
6. Write a short weekly report every Friday (template in Appendix D).

0.3 The twelve golden rules

1. **Never skip a stage or a gate.** Skipping is how integrations fail in production.
2. **Verify, never assume.** Do not assume a vendor has an API, allows your use, or will not change it. Check documentation, permissions, licences and limits.
3. **Choose the simplest architecture that works.** A scheduled script beats a message broker if the requirement is one nightly sync.
4. **Never bypass access restrictions.** If no approved API exists, we propose approved alternatives. We do not scrape behind logins or defeat protections.
5. **Never put a secret or client data into Git.** Not once, not "temporarily". Use `.env` locally and a secrets manager in staging and production.
6. **Test the failure, not only the success.** Duplicate events, timeouts, rate limits and vendor downtime are normal events, not rare ones.
7. **Every integration is idempotent.** Running the same message twice must not create two invoices.
8. **No test, no merge.** Code without an automated test does not enter the main branch.
9. **Write it down.** If it is not in a document or a ticket, it did not happen.
10. **Least privilege.** Request the smallest access scope that does the job.
11. **Ask early.** Being stuck for 30 minutes without asking is a mistake, not a virtue.
12. **Client data belongs to the client.** Treat it as if it were your own bank details.

0.4 Department map at a glance



Stages 01 to 06 are **pre-sales and design** (no code is written for the client). Stages 07 to 13 are **build and release**. Stages 14 to 16 are **run and support**, which is where recurring revenue and client trust are earned.

0.5 Who is who (plain words)

Role	What they really do	You ask them when
Founder	Decides what we sell, signs contracts and prices	You need a decision about money, scope or clients
Project Manager (PM)	Plans the work, tracks time, speaks to the client daily	You need priorities, deadlines or client contacts
API Architect	Chooses the architecture and reviews designs	You are unsure which integration pattern to use
Senior Integration Engineer	Reviews your code, mentors you, owns hard problems	You are stuck technically or need a code review
Integration Engineer / Developer	Builds APIs, connectors and transformations	Day-to-day coding questions
Database Engineer	Designs schemas, migrations, backup and recovery	You need a table, an index or a data model
Security Engineer	Reviews authentication, secrets and OWASP risks	Anything touching tokens, keys, tenants or personal data
QA Engineer	Tries to break the software before the client does	You want to know how to test something
DevOps Engineer	Pipelines, containers, servers, monitoring, alerts	Deployment, environments, backups, dashboards
Business Analyst	Writes down how the client's process really works	You do not understand the client's workflow
Training Manager	Runs onboarding, assessments and gate reviews	Your learning plan, assessments, mentor allocation

In a small startup one person may hold several roles. The responsibilities still exist and must still be covered.

Part 1: The Department: Mission, Services and Boundaries

1.1 Mission

> We connect your existing business applications so your teams can exchange data automatically, reduce manual entry and maintain consistent records.

The department exists because most businesses already own good software, but that software does not talk to each other. People re-type data, spreadsheets drift out of date and reports disagree. Our work is to make systems **exchange data reliably, securely and observably**.

1.2 Service portfolio

Line	Services	Typical client outcome
A. API development	REST APIs, GraphQL APIs, webhooks, internal / public / partner APIs, documentation, versioning, maintenance	A product or service exposes clean, documented, secure interfaces
B. System integration	CRM, ERP, HRMS, accounting, payment gateway, e-commerce, inventory, email and communication, cloud and third-party SaaS integration	Two or more business systems share data without manual re-entry
C. Integration automation	Data synchronisation, automated transfer, workflow and event-driven integration, scheduled jobs, document transfer, cross-platform automation	Repetitive hand-offs run on their own, with an audit trail
D. Enterprise integration	API gateways, middleware, message queues, event streaming, microservices communication, enterprise integration architecture, legacy integration	Many systems communicate through a governed, scalable backbone
E. Managed API services	Monitoring, integration maintenance, error monitoring, performance tuning, security reviews, version management, support and incident handling	The integration keeps working after go-live, and the client has one number to call

1.3 What we do and what we do not do

We do	We do not
Verify that a vendor offers an API and that the client's licence allows our use	Assume an API exists or "try it and see" on a client's production tenant
Use official, documented, versioned vendor APIs	Scrape pages behind a login, defeat captchas or bypass rate limits

We do	We do not
Store credentials in environment variables and secrets managers	Keep secrets in code, tickets, chats or the management portal
Offer approved alternatives when no API exists (file export, database read replica with consent, vendor professional services, RPA only if the vendor terms permit)	Promise a delivery date before feasibility (Stage 04) is signed off
Document every field mapping and failure behaviour	Ship an integration without monitoring and a runbook
State assumptions and risks in writing	Claim a guarantee the vendor's API cannot support

1.4 The "simplest thing that works" principle

New engineers over-build. Use this ladder and stop at the **first** rung that satisfies the written requirement:

Rung	Pattern	Use when	Typical tools
1	Direct API call (request/response)	Two systems, low volume, user waits for the answer	`requests` / `httpx`, vendor SDK
2	Scheduled synchronisation	Data may be minutes or hours old; vendor has no webhooks	Cron, scheduler, n8n schedule trigger
3	Webhook receiver	Vendor pushes events; you must react quickly	FastAPI / Express endpoint plus signature check
4	Queue between producer and consumer	Bursts, slow consumers, need retry and back-pressure	RabbitMQ, cloud queues (SQS, Service Bus, Pub/Sub)
5	Middleware / integration service	Three or more systems, shared transformations, central monitoring	Own service, n8n, Make where suitable
6	Event streaming	Very high volume, multiple independent consumers, replay needed	Kafka
7	API gateway in front of everything	Many consumers need uniform auth, rate limits, analytics	Kong, cloud-native gateways

> **Rule:** write the justification for moving up a rung in the architecture document (Stage 05). "It is more modern" is not a justification. A missed requirement (for example, "must survive 30 minutes of vendor downtime without losing events") is.

1.5 Reference architecture (what a typical VEL integration looks like)

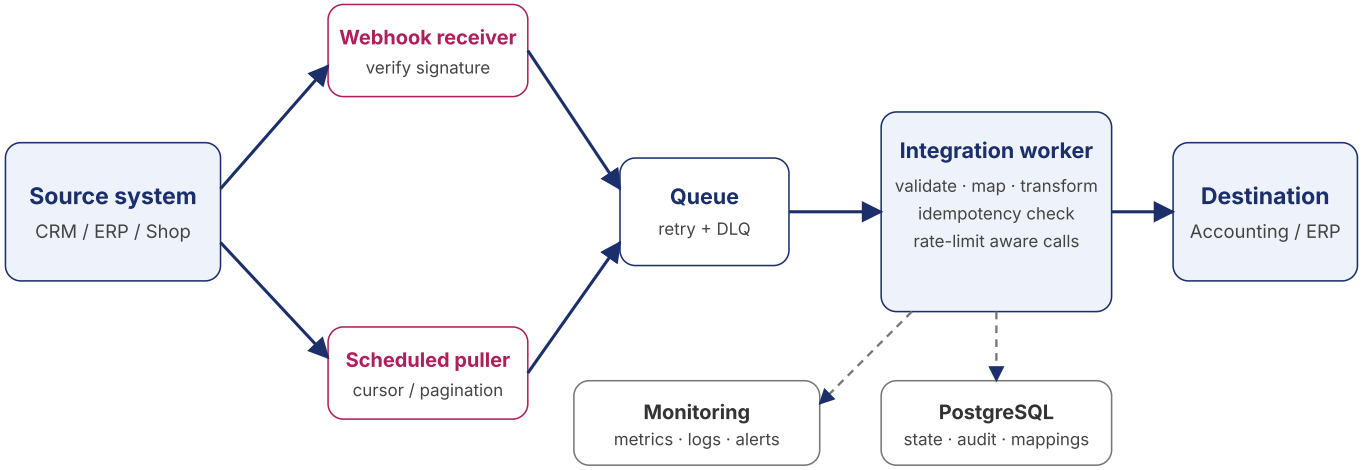


Figure 1: Reference integration architecture. Use only the parts the requirement needs.

1.6 Success measures for the department

Measure	Target (to be confirmed by the Founder)
Integrations delivered with a signed test report	100%
Production secrets found in Git	0
Integrations with monitoring and a runbook at go-live	100%
Duplicate-record incidents caused by retries	0
Critical incident acknowledgement	Per the SLA in the client's maintenance agreement
Trainee reaching supervised-project status within 90 days	Tracked; reasons recorded if not met

Part 2: Roles, Responsibilities and Career Path

2.1 Employee responsibilities (all grades)

Every employee in the department must:

- Follow this handbook and the company standards in Part 4.
- Keep **all client information confidential**, including after leaving the company.
- Use only company-approved tools and accounts. Never use personal accounts for client work.
- Never store credentials, tokens or personal data in Git, chat, email, tickets, screenshots or the portal.
- Record time and update ticket status daily.
- Write tests and documentation as part of the work, not after it.
- Report incidents, mistakes and near-misses **immediately**. Reporting early is rewarded; hiding is a disciplinary matter.
- Request a code review before every merge.
- Raise a risk in writing when a requirement, vendor or timeline looks unsafe.

2.2 Responsibility matrix (RACI)

R = Responsible (does it) · **A** = Accountable (signs off) · **C** = Consulted · **I** = Informed

Activity	Founder	PM	Architect	Sr. Engineer	Engineer	Security	QA	DevOps
01 Lead qualification	A	R	C	I	I			
02 Client discovery	I	A	R	C	R	C		
03 Requirements freeze	A	R	C	C	C	C	C	
04 Feasibility assessment	I	A	C	R	R	C		
05 Architecture design	I	C	A/R	C	C	C		C
06 Cost estimation	A	R	C	C				C
07 Environment setup		I		A	R	C		R
08 Repository structure		I	C	A	R			C
09 API development		I	C	A	R	C	C	
10 System integration		I	C	A	R	C	C	
11 API security	I	I	C	C	R	A	C	C
12 Testing and QA	I	I		C	R	C	A	
13 CI/CD and deployment	I	A	C	C	C	C	C	R

Activity	Founder	PM	Architect	Sr. Engineer	Engineer	Security	QA	DevOps
14 Production monitoring	I	I	C	C	C			A/R
15 Client handover	I	A	C	R	R	C	C	C
16 Maintenance	I	A	C	R	R	C		R

2.3 Role cards

Trainee API Engineer (days 1 to 90)

Does: completes the 90-day roadmap, builds the ten practical projects, shadows live projects. **May not:** deploy to production, access client production systems, or hold client credentials. **Reports to:** assigned mentor (Senior Integration Engineer).

Junior Integration Engineer

Does: implements well-specified endpoints and connectors under review; writes tests and docs. **May:** work in staging with client sandbox credentials issued through the access register. **Promotion test:** two supervised projects delivered with no critical defect found by the client.

Integration Engineer

Does: owns a connector end-to-end, writes the data-mapping document, handles vendor liaison. **Promotion test:** designs and delivers a multi-system integration with architect review.

Senior Integration Engineer

Does: reviews all code, mentors trainees, leads feasibility (Stage 04), owns hard incidents.

API Architect

Does: owns architecture (Stage 05), the API governance standards, the API inventory and the technology radar.

2.4 Mentor responsibilities

Each trainee has one named mentor who must: hold a 30-minute weekly one-to-one, review every pull request within one working day, mark the weekly assessment, and recommend pass / repeat at each gate. A mentor supervises at most three trainees at once.

2.5 Separation of duties (important for client trust)

Rule	Why
The author of code never approves their own pull request	Independent review catches mistakes

Rule	Why
The person who deploys to production is not the only person who approved it	Prevents unreviewed changes reaching clients
Production credentials are held by the DevOps/Security owner, never by trainees	Limits blast radius
Billing changes need PM and Founder approval	Prevents revenue leakage and disputes

VEL
SOFTWARE SOLUTIONS

Part 3: The Corporate Workflow: 16 Stages from Lead to Maintenance

3.0 How the workflow runs

Each stage has an **owner**, **inputs**, **steps**, a **deliverable** and a **gate**. A gate is a written sign-off. If the gate fails, the project stays in the stage.

#	Stage	Owner	Main deliverable	Gate
01	Lead qualification	PM	Lead qualification report	G1 Go / no-go to discovery
02	Client discovery	Architect	Discovery notes + questionnaire answers	G2 Discovery complete
03	Requirements freeze	PM	Integration specification v1.0	G3 Client signs spec
04	API feasibility	Sr. Engineer	API feasibility report	G4 Feasible / conditional / not feasible
05	Architecture design	Architect	Architecture document + diagrams	G5 Architecture approved
06	Cost estimation	PM	Costed estimate, quotation, SOW	G6 Founder approves price
07	Environment setup	DevOps	Dev, staging and production environments	G7 Environments verified
08	Repository structure	Sr. Engineer	Repository from the VEL template	G7 (same)
09	API development	Engineer	Working, tested, documented API	G8 Code review + tests green
10	System integration	Engineer	Working connectors with retry and idempotency	G8
11	API security	Security	Security review report	G9 No open high-severity issue
12	Testing and QA	QA	Test report	G9
13	CI/CD and deployment	DevOps	Staging live; production release approved	G10 Production release approval
14	Production monitoring	DevOps	Dashboards, alerts, incident process live	G10
15	Client handover	PM	Handover pack + signed acceptance	G11 Client acceptance

#	Stage	Owner	Main deliverable	Gate
16	Maintenance	PM / Sr. Engineer	Monthly support and change log	G12 Quarterly service review

---

Stage 01: Lead Qualification

Goal: decide quickly and honestly whether this is a project we can deliver profitably.

Capture:

Item	Question to answer
Client industry	Which sector? Any regulation (healthcare, finance)?
Existing systems	Names, versions, cloud or on-premises
Integration requirements	What should connect to what, and why?
Business problem	What does the problem cost them today (hours, errors, delays)?
Number of applications	Two, or a web of ten?
Expected data volume	Records per day / month; peak bursts
Required automation	Real-time, hourly, daily? Manual approval steps?
Budget	Range or ceiling; who approves?
Timeline	Hard deadline and its reason

Output: Lead qualification report (template, Appendix A). Score the lead red / amber / green on: clear problem, reachable decision-maker, API likelihood, budget fit, timeline realism.

Gate G1: PM and Founder agree to invest discovery time.

Stage 02: Client Discovery

Goal: understand the real process, not the assumed one.

Discovery checklist (the full questionnaire is in Appendix B):

- **Source systems** and **destination systems**: product, edition, version, hosting.
- **API availability:** does a documented API exist? Which edition or plan unlocks it?
- **Vendor documentation:** link, version, last-updated date.
- **Authentication method:** API key, OAuth 2.0, basic auth, certificates, VPN.
- **Data formats:** JSON, XML, CSV, flat files, EDI.
- **Rate limits** and quotas, per minute, per day, per tenant.
- **Existing workflows:** who does the task today, in which tool, how often.
- **Data ownership:** who is the system of record for each entity?
- **Failure scenarios:** what happens today when the data is wrong or late?
- **Expected frequency** and peak load.

- **Compliance:** personal data, financial data, health data, data residency, retention.

> **Do not assume** a third-party platform exposes an API or allows unrestricted integration. Verify **access, permissions, licensing and vendor limitations** in writing. A client saying "yes, it has an API" is a lead, not a fact.

Gate G2: every question answered or recorded as an open risk with an owner.

Stage 03: Integration Requirements Freeze

Goal: one written specification that both sides sign. Changes after this point go through change control.

The integration specification must contain (template in Appendix C):

Section	Content
Source and destination	Application, version, environment
Data entities	e.g. Customer, Order, Invoice, Payment
Fields and mapping rules	Source field, destination field, type, required, transformation
Trigger conditions	Event, schedule or manual
Frequency	Real-time, every N minutes, nightly
Transformations	Currency, date format, unit, code lookup, merge/split
Authentication	Method per system, token lifetime, who rotates credentials
Error handling	Which errors are retried, which go to manual review
Retry policy	Attempts, back-off, maximum age
Expected response	Status codes and body for every call
Idempotency	The key that makes repeats safe
Logging and monitoring	What is logged (never secrets), what alerts fire
Acceptance criteria	Measurable pass conditions

Gate G3: client signs specification v1.0. Further changes follow the change-request process.

Stage 04: API Feasibility Assessment

Goal: prove, with evidence, that the specification can be built.

Check	Evidence required
API documentation	Link and version recorded
API availability	Works in the client's licence tier

Check	Evidence required
Access permissions	Credentials or app registration approved in writing
Authentication	Test token obtained in a sandbox
API version	Current version; support end date
Rate limits	Documented numbers; compare with expected peak
Pagination	Style (offset, cursor, page token) and maximum page size
Webhook support	Events available; signature method; retry behaviour
Data limits	Payload size, field length, attachment limits
Vendor costs	API subscription, per-call fees, partner programme fees
Sandbox availability	Test environment exists; data realism
Deprecation policy	Notice period for breaking changes

If no API exists: assess **approved alternatives** and report their reliability honestly: vendor-supported file export/import, a vendor-provided integration marketplace, database access *only with the vendor's and client's written consent*, or a vendor professional-services engagement. **Do not bypass access restrictions.**

Output: API feasibility report with one verdict per integration: **Feasible**, **Feasible with conditions**, or **Not feasible**.

Gate G4: Senior Engineer and Architect sign. Not-feasible items are removed from scope or re-planned.

Stage 05: Architecture Design

Deliverables: system context diagram · API architecture · data-flow diagram · authentication flow · database architecture · integration sequence diagram · error-handling flow · monitoring architecture.

Pattern selection table (choose one, justify it):

Requirement	Best-fit pattern	Why
User waits for the answer; low volume	**Direct API integration**	Simple, immediate, easy to debug
Vendor has no webhooks; data can be minutes old	**Scheduled synchronisation**	Predictable load, easy retries
Vendor pushes events; fast reaction needed	**Webhook** (with signature verification and quick acknowledgement)	Low latency, low polling cost
Bursts, slow destination, must not lose events	**Message queue** (RabbitMQ or cloud queue)	Buffering, retry, dead-letter handling
3+ systems, shared transformations	**Middleware**	One place for mapping, monitoring and security

Requirement	Best-fit pattern	Why
Many consumers need the same events; replay	**Event-driven / streaming** (Kafka)	Decoupling, replay, scale

Always decide and document: the system of record for each entity · conflict rule when both sides edit · idempotency key · retry/dead-letter behaviour · what "eventually consistent" means for this client (how many seconds or minutes of lag are acceptable) · why **distributed transactions are avoided** (use sagas or compensating actions, and say so in writing).

Gate G5: Architect and Security sign. Client sees the diagrams.

Stage 06: Cost Estimation

Distinguish **one-time**, **recurring** and **usage-based** costs. Use **official vendor pricing pages**, recorded with date and link. Never quote from memory.

Cost line	Type	Basis
Discovery	One-time	Hours × blended rate
API development	One-time	Endpoints × complexity
Integration development	One-time	Connectors × entities × complexity
Database	Recurring	Hosting tier + backup
API subscriptions	Recurring / usage	Vendor price sheet
Vendor licences	Recurring	Vendor price sheet
Cloud infrastructure	Recurring	Calculator from cloud provider
Message broker	Recurring	Managed service or self-host effort
Monitoring	Recurring	Tool plan or self-host effort
Testing and documentation	One-time	Hours
Maintenance and support	Recurring	Retainer hours

Per-integration cost = (all one-time costs for that integration) + 12 × (its monthly recurring costs). **Per-execution cost** = (variable vendor fees + compute + queue + log storage per run).

Worked pricing and margin example: Part 9. **Gate G6:** Founder approves quotation and SOW.

Stage 07: Development Environment Setup

Install and verify: **VS Code · Python 3.12+ or Node.js LTS · Git · GitHub account · Postman (and/or Insomnia) · PostgreSQL · Docker · required packages · API testing tools.** Detailed commands: Part 5.

Maintain **three separated environments**:

	Development	Staging	Production
Purpose	Build and experiment	Rehearse the release with client sandbox	Real client data
Data	Fake / sample only	Anonymised or sandbox	Real
Credentials	Personal sandbox	Staging secrets store	Production secrets store, restricted
Who may deploy	Developer	CI pipeline	CI pipeline with human approval

Gate G7: each environment starts from a clean checkout using the README alone.

Stage 08: Professional Repository Structure

Use the VEL template. Adapt folder names to the framework, but keep the intent.

```
api-integration-project/
│
├── README.md                  # What this is, how to run, how to test, who to ask
├── .gitignore                 # Keeps .env, venvs, caches, build output out of Git
├── .env.example               # Names of every variable, with fake values. Real .env never
committed
│
├── docs/                      # Everything a new engineer or the client needs to understand the
system
│   ├── project-overview.md    # Business problem, scope, stakeholders
│   ├── requirements.md        # Frozen specification and change log
│   ├── api-specification.md   # Human-readable API behaviour (generated reference lives in api-
docs/)
│   ├── architecture.md        # Diagrams and the reasons for each choice
│   ├── data-mapping.md        # Field-by-field mapping and transformation rules
│   ├── authentication.md      # How each system authenticates; rotation process (no secrets)
│   ├── error-handling.md      # Error catalogue, retry rules, manual-review queue
│   ├── deployment.md          # Release, rollback, environment variables
│   └── troubleshooting.md     # Symptom → cause → fix, built from real incidents
│
├── src/
│   ├── api/
│   │   ├── routes/            # URL paths and HTTP verbs only; no business logic
│   │   ├── controllers/       # Translate HTTP to service calls and back
│   │   ├── services/          # Business logic; no HTTP concepts
│   │   └── schemas/           # Request/response models and validation
│   │
│   ├── integrations/          # One sub-folder per external system
│   │   ├── crm/               # Client for the CRM: auth, calls, mapping
│   │   ├── erp/
│   │   ├── payments/
│   │   └── email/
│   │
│   ├── authentication/        # Token validation, API keys, OAuth flows, RBAC
│   ├── middleware/            # Request IDs, logging, rate limiting, CORS, error handling
│   ├── database/              # Models, repositories, migrations
│   ├── transformations/       # Pure functions that map one system's data to another's
│   ├── webhooks/              # Inbound receivers, signature checks, replay protection
│   ├── messaging/             # Producers, consumers, retry and dead-letter handling
│   ├── monitoring/            # Metrics, health checks, tracing setup
│   └── utils/                 # Small shared helpers (time, ids, hashing)
│
├── tests/
```

```
│   ├── unit/                  # Pure logic: mappings, validators
│   ├── integration/           # Real database/queue, mocked or sandbox vendor
│   ├── contract/              # Checks our calls and responses match the OpenAPI / vendor contract
│   ├── security/              # Authn/authz, injection, rate-limit and tenant-isolation tests
│   └── performance/           # Load and soak tests
│
├── deployment/
│   ├── docker/                # Dockerfile, docker-compose for local and staging
│   ├── nginx/                 # Reverse proxy, TLS, headers, limits
│   └── scripts/               # Release, rollback, backup, restore scripts
│
├── api-docs/
│   └── openapi.yaml           # Machine-readable contract; source for Swagger UI and contract tests
│
├── monitoring/
│   └── alerts/                # Alert rules (error rate, queue depth, auth failures, vendor down)
│
└── .github/
    └── workflows/             # CI (tests, lint, scan, build) and approval-gated CD
```

Layering rule: *routes → controllers → services → integrations/database.* A lower layer never imports from a higher layer. Transformations are pure functions with no I/O, so they can be unit-tested exhaustively.

Never commit real credentials, tokens, private keys or client data. A pre-commit secret scan and the CI secret scan both run on every push.

Stage 09: API Development

Implement, in this order, for each resource: **schema → test → route → service → persistence → error responses → logging → documentation.**

Must be present: routes · request validation · response schemas · business logic · database operations · authentication · authorisation · consistent error responses · structured logging · API documentation. Follow the company standards in Part 4. Assignment 01 (Part 8) is your first complete example.

Stage 10: System Integration

For every external connector implement: **authentication → extraction → mapping → transformation → validation → synchronisation → webhook handling → retry → idempotency → error queue.**

Handle these as normal conditions:

Situation	Required behaviour
HTTP 429 (rate limited)	Honour `Retry-After`; slow down; never hammer
HTTP 5xx / timeout	Retry with exponential back-off and jitter; give up after the policy limit
HTTP 4xx (not 429)	**Do not retry blindly**: record, route to manual review
Partial batch failure	Process good records; park bad ones with the reason
Vendor downtime	Queue and replay on recovery; alert if outage exceeds threshold
Duplicate event	Detected by idempotency key; processed once

Situation	Required behaviour
Out-of-order events	Use version/timestamp; ignore stale updates
Token expiry	Refresh automatically; alert on repeated refresh failure

---

Stage 11: API Security

Apply the **OWASP API Security Top 10 (2023)** as the review checklist:

Code	Risk	Our standard control
API1	Broken Object Level Authorization	Every object access checks the caller's right to *that* object
API2	Broken Authentication	Short-lived tokens, secure key storage, lockout and alert on repeated failure
API3	Broken Object Property Level Authorization	Explicit response schemas; never return whole database rows
API4	Unrestricted Resource Consumption	Rate limits, page-size caps, payload limits, timeouts
API5	Broken Function Level Authorization	Role / scope check on every endpoint, deny by default
API6	Unrestricted Access to Sensitive Business Flows	Throttle and monitor high-value flows (payments, exports)
API7	Server Side Request Forgery	Allow-list outbound hosts; validate user-supplied URLs
API8	Security Misconfiguration	Hardened defaults, no debug in production, strict CORS, security headers
API9	Improper Inventory Management	Maintain the API inventory; retire old versions
API10	Unsafe Consumption of APIs	Validate data from vendors as strictly as user input

Also required: **HTTPS only · authentication · authorisation · input validation · output filtering · rate limiting · CORS restrictions · secrets management · audit logging · token expiry · least privilege · tenant isolation where applicable.**

Stage 12: Testing and QA

Test **failure scenarios, not just successful calls.** The test report must show a result for each:

#	Area	Example test
1	API functionality	Each endpoint returns the documented response
2	Request validation	Missing/invalid fields give 422 with field names
3	Authentication	No/expired/forged token gives 401

#	Area	Example test
4	Authorisation	User A cannot read user B's object (403/404)
5	Data accuracy	1,000 sample records map exactly as per the mapping document
6	Integration functionality	End-to-end source → destination in staging
7	Duplicate event handling	Same webhook delivered 3× creates 1 record
8	Retry handling	Destination fails twice then succeeds; one record
9	Rate limits	Vendor 429 triggers slow-down, no data loss
10	Vendor downtime	Destination off for 30 minutes; backlog drains on recovery
11	Performance	Meets agreed p95 latency at expected peak
12	Security	OWASP checklist, dependency and secret scan clean
13	Logging	Every request traceable by request ID; no secrets in logs
14	Recovery	Restore from backup; replay from dead-letter queue

---

Stage 13: CI/CD and Deployment

Step	Content
Git workflow	`main` is protected. Work on `feature/*` branches. Pull request + one approval + green CI. Conventional commit messages
Automated tests	Unit, integration and contract tests on every push
Code quality	Formatter, linter, type check (e.g. `ruff` , `mypy` , or ESLint + `tsc`)
Security scans	Dependency vulnerability scan, secret scan, container image scan
Docker build	Reproducible, minimal, non-root image tagged with the commit
Staging deployment	Automatic on merge to `main`
Production release	**Manual approval** by PM/Founder after staging sign-off
Rollback	One documented command to redeploy the previous image; database migrations are backward-compatible for one release

---

Stage 14: Production Monitoring

Monitor and alert on: **API response times · error rates · request volumes · failed synchronisations · authentication failures · rate-limit responses · queue depth · retry counts · webhook failures · vendor availability.**

Signal	Example alert
Error rate	5xx above agreed threshold for 5 minutes
Latency	p95 above target for 10 minutes
Failed syncs	Any record in dead-letter queue older than 1 hour
Queue depth	Growing for 15 minutes
Auth failures	Spike versus baseline
Vendor availability	Health probe failing

Tools: **Prometheus + Grafana** for metrics, **OpenTelemetry** for traces/metrics/logs instrumentation, a suitable log platform for search. Generate an **incident report** for every Sev-1/Sev-2 (template, Appendix F).

Stage 15: Client Handover

Deliver and obtain a signature on: API documentation · integration architecture · data mapping document · authentication guide · deployment guide · error handling guide · monitoring guide · troubleshooting guide · **test report** · **access register** (who has which access; no secrets) · **maintenance agreement**. Checklist: Appendix G.

Stage 16: Maintenance

Provide: API version upgrades · **vendor API change monitoring** (subscribe to vendor changelogs and deprecation notices) · integration error resolution · security updates · performance optimisation · monitoring · documentation updates · monthly support report. Review SLAs with the client each quarter (Gate G12).

Part 4: API Governance and Company Standards

These standards apply to every API VEL builds. A deviation needs a written exception approved by the API Architect.

4.1 Naming

Item	Rule	Good	Bad
Resource paths	Plural nouns, lowercase, hyphens	`/v1/purchase-orders`	`/getPurchaseOrder`, `/v1/Purchase`
Actions	HTTP method, not verb in path	`POST /v1/orders`	`/v1/createOrder`
Sub-resources	Nest one level	`/v1/clients/12/contacts`	`/v1/clients/12/contacts/5/notes`
JSON fields	`snake_case`, consistent across the API	`created_at`	`CreatedAt`, `createdAt` mixed
Dates	ISO 8601 in UTC	`2026-11-04T09:30:00Z`	`04/11/26`
Money	Integer minor units + currency code	`{"amount":125000,"currency":"INR"}`	`1250.00` as float
Identifiers	Opaque, stable	`"id": "cus_8f3a..."`	Sequential ids that leak volume (for pu

4.2 Versioning

- Version in the path: `/v1/...`. A new major version only for **breaking** changes.
- **Non-breaking** (adding optional fields, new endpoints) stays in the same version.
- **Breaking:** removing/renaming fields, changing types or meaning, tightening validation, changing auth.
- Run at most **two** major versions in production. Announce the old one's end date at least the notice period promised in the client's agreement.

4.3 Authentication

Method	Use when	Do not use when
API key (header `X-API-Key`)	Server-to-server, one known partner, low sensitivity	End-user identity is needed; the key could end up in a browser or mobile app
OAuth 2.0 (client credentials)	Machine-to-machine with scopes and rotation	Simple internal tool where a key is enough
OAuth 2.0 + OpenID Connect (authorisation code with PKCE)	Acting on behalf of a user; single sign-on	Pure back-office batch jobs
JWT access tokens	Stateless validation across services; short life (minutes)	You need instant revocation and cannot afford a deny-list or short expiry
Session cookie	Browser applications of our own	Third-party API consumers

Rules: keys sent in **headers only**, never in URLs · tokens expire · refresh tokens stored securely and rotated · every key/token has an owner and a rotation date in the access register · compare secrets with constant-time functions.

4.4 Error response standard

Every error uses this shape (Assignment 01 implements it):

```
{
  "error": {
    "code": "validation_error",
    "message": "Request validation failed",
    "details": [{"field": "email", "problem": "String should match pattern"}],
    "request_id": "7c1e1f1e-3a3c-4c4a-9a4e-0c7f4d6b2a11"
  }
}
```

Status	Meaning	Use for
200	OK	Successful read / update
201	Created (+ `Location` header)	Successful create
202	Accepted	Work queued, not finished (webhooks, async jobs)
204	No content	Successful delete
400	Bad request	Malformed syntax
401	Unauthorised	Missing/invalid credentials
403	Forbidden	Valid credentials, not allowed
404	Not found	Unknown resource (also used to hide existence from unauthorised callers)
409	Conflict	Duplicate, version conflict

Status	Meaning	Use for
422	Unprocessable	Validation failed
429	Too many requests (+ `Retry-After`)	Rate limit
500 / 502 / 503 / 504	Server / upstream error	Our fault or upstream unavailable

Never return stack traces, SQL, internal paths or secrets in an error body.

4.5 Pagination, filtering, sorting

- Every list endpoint is paginated. Default `page_size` 20, maximum 100.
- Small, stable data: `page` + `page_size` and a `total`. Large or fast-changing data: **cursor-based** (`next_cursor`).
- Filters are query parameters named after the field: `?status=active&industry=retail`.
- Sorting: `?sort=name` ascending, `?sort=-created_at` descending; allow-list the sortable fields.

4.6 Documentation

- Every API has an **OpenAPI 3** description kept in `api-docs/openapi.yaml` (generated or hand-written, but always in the repository).
- Every endpoint has a summary, request/response examples and the error cases.
- Documentation is part of "done". A pull request that changes behaviour without changing docs is rejected.

4.7 Logging

- **Structured** (JSON in staging/production), one line per request: time, level, request ID, method, path, status, duration, caller id.
- Propagate the `X-Request-ID` header to every downstream call so one failure can be traced end-to-end.
- **Never log:** passwords, tokens, API keys, full card numbers, national IDs, health data, full request bodies of sensitive endpoints.

4.8 Security (summary)

HTTPS only · deny by default · validate every input · filter every output · least privilege · rate limit · strict CORS allow-list · secrets from environment/secrets manager · audit log for create/update/delete and authentication events · dependency and secret scanning in CI.

4.9 Testing

Every endpoint: success, validation failure, unauthenticated, unauthorised, not-found. Every connector: success, retry, duplicate, rate-limit, vendor-down. Minimum automated coverage and the contract test baseline are set per project in the test plan.

4.10 Release management

Semantic versioning for the service (`MAJOR.MINOR.PATCH`) · changelog entry per release · staging soak before production · production release by approval only · rollback tested at least once per project.

4.11 Deprecation

1. Announce in writing with a date. 2. Add `Deprecation` and `Sunset` response headers on the old version. 3. Provide a migration guide. 4. Monitor remaining callers. 5. Remove only after the notice period and with zero (or accepted) traffic.

4.12 Client access

Each client consumer gets its own credentials, scopes and rate limit. Access is requested by ticket, approved by the PM, recorded in the access register, reviewed every quarter and revoked on contract end or staff change.

4.13 API inventory

The Architect maintains one inventory of every API and integration. Fields:

Field	Example
API / integration name	Client Registry API
Owner (person + team)	Senior Integration Engineer, API team
Version	v1.0.0
Environment(s)	dev, staging, prod
Consumers	CRM sync worker, client portal
Authentication	API key (rotation due 2027-01-15)
Data classification	Internal / Confidential / Personal
Lifecycle status	Planned · Active · Deprecated · Retired
Documentation link	Repository `api-docs/openapi.yaml`
Last security review	Date

Part 5: Beginner Onboarding Programme

5.1 Day-0 checklist (before the first morning)

Item	Owner	Done
Company email account created	Admin	☐
GitHub account invited to the VEL organisation (two-factor authentication **on**)	Admin	☐
Password manager account set up	Security	☐
Laptop with disk encryption, OS updates, screen lock	DevOps	☐
Mentor assigned and introduced	Training Manager	☐
Signed confidentiality agreement and acceptable-use policy	Founder	☐
Access register entry created (least privilege: training sandbox only)	Security	☐

5.2 Week-1 schedule

Day	Morning	Afternoon	Output
1	Welcome, company tour, read Part 0 and Part 1	Security briefing; set up password manager and 2FA	Signed policies
2	Read Part 3 (workflow overview)	Install tools (5.3)	Screenshot of every tool's version
3	Level 1 lesson: software fundamentals	Level 1 quiz; first Git commit to the training repo	Quiz ≥ 80%
4	Level 2 lesson: HTTP and the web	Hands-on: inspect requests in browser dev tools and Postman	Notes file
5	Python starter exercises	Weekly review with mentor; Friday report	First weekly report

5.3 Install the toolchain

Run each verification command and keep the output in your notes.

Tool	Install	Verify
VS Code	Official installer from code.visualstudio.com; add extensions *Python*, *Pylance*, *REST Client* (optional), *Docker*, *YAML*, *GitLens*	Open a folder; run `code --version`
Python 3.12+	python.org installer or OS package manager	`python --version`

Tool	Install	Verify
Node.js LTS (needed from Level 6)	nodejs.org installer	`node --version` and `npm --version`
Git	git-scm.com	`git --version`
GitHub	Create/sign in with the company account; enable 2FA; add an SSH key	`ssh -T git@github.com`
Postman and/or **Insomnia**	Official downloads	Send a `GET` to a public test API
PostgreSQL 16 (use Docker if easier)	Official installer or `docker run` (see Assignment 02)	`psql --version`
Docker	Docker Desktop or Docker Engine	`docker run hello-world`

Configure Git once:

```
git config --global user.name  "Your Name"
git config --global user.email "your.name@company-domain"
git config --global init.defaultBranch main
git config --global pull.rebase true
```

Create a virtual environment for every project (never install packages globally):

```
python -m venv .venv
source .venv/bin/activate        # Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

5.4 Security rules for every day

1. Two-factor authentication on every account (email, GitHub, cloud, password manager).
2. Credentials live **only** in the password manager or the secrets store.
3. Never paste a client token, key or record into a public AI tool, forum, chat or ticket.
4. Lock your screen when you leave your desk. Disk encryption stays on.
5. Report a lost device, a leaked key or a suspicious email **immediately**.
6. Use test data. If you must see real client data, a Senior Engineer approves it in writing first.

5.5 Working agreement

Topic	Rule
Stand-up	10 minutes, daily: yesterday / today / blocked
Tickets	Every task is a ticket; move it as you work
Commits	Small, frequent, clear message: `type(scope): what and why`
Pull requests	Template completed; tests green; reviewer assigned

Topic	Rule
Help	Try for 30 minutes, then ask. Include what you tried and the exact error
Friday report	Done, learning, blocked, next week (Appendix D)

5.6 First-week success criteria

By the end of week 1 you can: explain client-server, request and response in your own words · send a `GET` and a `POST` in Postman · read a JSON document · create a Git branch, commit and open a pull request · state the twelve golden rules.

Part 6: The 90-Day Learning Roadmap

The roadmap has **three phases**, three **gates**, and **ten practical projects**. A gate must be passed before the next phase starts. If you do not pass, you repeat the weakest module with your mentor (one repeat is normal).

6.1 Roadmap at a glance

Phase	Days	Weeks	Levels	Projects	Gate
1 Foundations	1 to 30	1 to 4	1 · 2 · 3 · start of 4	P1 Basic REST API	**Gate 1**
2 Build and secure	31 to 60	5 to 9	4 · 5 · 6 · 7	P2 API + PostgreSQL · P3 Authentication · P4 Integrate two sample apps	**Gate 2**
3 Integrate and operate	61 to 90	10 to 13	8 · 9	P5 Webhook receiver · P6 Scheduled sync · P7 Retry + idempotency · P8 Monitoring dashboard · P9 Containerise · P10 Capstone	**Gate 3**

6.2 Week-by-week plan

Week	Focus	You will	Evidence
1	Onboarding, Levels 1 and 2	Install tools; learn client-server, HTTP, DNS, ports, TLS, headers, cookies, CORS	Week-1 criteria (5.6)
2	Level 3 Python I	Variables, functions, data structures, errors, JSON, files	20 exercises passed
3	Level 3 Python II + Level 4 start	Environment variables, logging, async basics; REST concepts, methods, status codes	Postman collection of a public API
4	Level 4 + **Project 1**	Build, test and document the Client Registry API (Assignment 01)	**Gate 1** review
5	Level 7 (SQL) + Level 6 (FastAPI deep dive)	PostgreSQL, tables, relationships, indexes; dependency injection, routers	SQL exercise set
6	**Project 2**	Replace the in-memory store with PostgreSQL, migrations (Alembic), connection pooling	Migration history in Git
7	Level 5 + **Project 3**	API keys, OAuth 2.0, JWT, RBAC, scopes, secrets; implement token auth	Security tests green
8	**Project 4 (part 1)**	Two sample apps (a "shop" and an "accounting" app); read vendor-style docs; design the mapping	Data-mapping document
9	**Project 4 (part 2)**	Build the integration; introduce NestJS / Express to compare	**Gate 2** review

Week	Focus	You will	Evidence
10	Level 8 + **Project 5**	Webhooks: receive, verify signature, acknowledge fast, process async	Replay-attack test
11	**Projects 6 and 7**	Scheduled sync with cursors; retries with back-off, idempotency keys, dead-letter queue (RabbitMQ)	Failure-scenario test report
12	Level 9 + **Projects 8 and 9**	OpenAPI, contract tests, load test; Prometheus/Grafana dashboard; Docker image and compose	Dashboard + image
13	**Project 10: Capstone**	Full simulated enterprise integration, Stages 03 to 15 in miniature, plus a demo	**Gate 3** board review

> **Daily rhythm (about 7 hours):** 1 h theory with the official source · 3 h hands-on build · 1 h testing and documentation · 1 h review or pairing · 1 h reading / reflection. Fridays: review, report, assessment.

6.3 The three gates

Gate 1 (end of week 4): "Can build a correct basic API"

Criterion	Pass mark
Quiz on Levels 1 to 4	≥ 80%
Assignment 01 delivered per the checklist in Part 8	All "must" items
Code review feedback addressed	Yes
Can explain a request/response cycle at a whiteboard	Yes

Gate 2 (end of week 9): "Can build a secure API and connect two systems"

Criterion	Pass mark
Quiz on Levels 5 to 7	≥ 80%
Projects 2, 3, 4	Reviewed and merged
OWASP API Top 10 oral check	Explain each risk and our control
No secret found in any repository	Zero

Gate 3 (end of week 13): "Ready for supervised commercial projects"

Criterion	Pass mark
Quiz on Levels 8 to 9	≥ 80%
Projects 5 to 9	Reviewed and merged

Criterion	Pass mark
Capstone (Project 10)	Demonstrated live; failure scenarios shown
Panel (Architect + Sr. Engineer + QA)	Recommends "supervised commercial project"
Communication check	Writes a client-ready technical summary

Outcome of Gate 3: *Junior Integration Engineer, supervised.* Works on a live project with a named senior reviewer, no production access, and weekly review.

6.4 Assessment rubric (every project)

Dimension	Weight	What "excellent" looks like
Correctness	25%	All requirements work; edge cases handled
Failure handling	20%	Retries, duplicates, timeouts, invalid data tested
Security	20%	No secrets in Git; auth enforced; input validated
Tests	15%	Automated, meaningful, run in CI
Documentation	10%	A stranger can run it from the README
Code quality	10%	Layered, readable, small functions, typed

Score each 0 to 4. **Pass** needs ≥ 70% overall **and** no zero in Security or Failure handling.

6.5 What "supervised commercial project" means

You may: implement tickets, write tests and docs, join client calls as an observer, work in staging with sandbox credentials. You may not: deploy to production, hold production credentials, commit to client dates, or change scope. Every pull request is reviewed by your named senior. After two clean projects you are considered for promotion to Integration Engineer (section 2.3).

Part 7: The Nine-Level Curriculum

Each level lists what you learn, why it matters to the business, the lab you do, and how you are assessed. Every lesson is delivered in the **16-point teaching format** (section 7.10). Always read the official source named for the level.

Level 1: Software Fundamentals

Topics	What is software · client-server architecture · frontend and backend · database · server · application · request and response · authentication · data exchange · JSON and XML
Business purpose	You cannot integrate systems you cannot describe. Clients explain problems in business words; you translate them to systems and data.
Real-world example	A shop website (client) asks the order server for today's orders; the server reads the database and replies with JSON.
Lab	Draw the client, server and database of an online shop. Open browser dev tools and find three requests and their JSON responses.
Official source	MDN Web Docs: *How the web works*
Assessment	20-question quiz (≥ 80%); one-page diagram explained aloud

Level 2: Networking and Web Fundamentals

Topics	HTTP · HTTPS · DNS · IP addresses · ports · methods · status codes · headers · cookies · TLS · CORS · reverse proxies
Business purpose	Most production integration faults are network, certificate, header or timeout faults.
Real-world example	A webhook fails because the client's firewall blocks port 443 from our server's IP.
Lab	`curl -v https://example.com`; read the TLS handshake, headers and status; resolve a name with `nslookup`; trigger and fix a CORS error in a test page.
Common errors	Mixing up 401 and 403 · certificate name mismatch · wrong port · missing `Content-Type`
Official source	MDN HTTP guide
Assessment	Practical: diagnose three broken requests from a worksheet

Level 3: Programming (Python first, TypeScript later)

Topics	Variables · functions · data structures (list, dict, set, tuple) · error handling · async programming · JSON processing · environment variables · logging (Python first; JavaScript/TypeScript introduced only when a project requires it)
Business purpose	All connectors are code that transforms data and handles failure.
Lab	Read a JSON file of 1,000 orders, validate fields, write a cleaned file and a rejects file with reasons; read configuration only from environment variables; log with levels.
Common errors	Mutable default arguments · swallowing exceptions · hard-coded secrets · printing instead of logging
Official source	docs.python.org tutorial
Assessment	20 graded exercises plus one code review

Level 4: API Fundamentals

Topics	What an API is · how APIs work · REST · GET · POST · PUT · PATCH · DELETE · request headers · request body · response body · status codes · query and path parameters · pagination · filtering · sorting · rate limits
Business purpose	These are the words clients and vendors use in every document.
Lab	Build the Postman collection for a public API; then **Assignment 01** (Part 8).
Key idea: safe and idempotent methods	`GET` is safe. `GET`, `PUT`, `DELETE` are idempotent (repeating changes nothing more). `POST` is not idempotent by default. `PATCH` depends on the design.
Official source	MDN HTTP methods; OpenAPI Initiative; FastAPI tutorial
Assessment	Assignment 01 and Gate 1

Level 5: API Authentication and Security

Topics	API keys · OAuth 2.0 · OpenID Connect · JWT · sessions · RBAC · scopes · token expiry · refresh tokens · secrets management · TLS · input validation · rate limiting
Business purpose	A leaked key or a missing authorisation check is a client-losing incident.
Selection guide	See section 4.3. Rule of thumb: API key for one trusted server; OAuth 2.0 client credentials for machine-to-machine with scopes; authorisation code + PKCE with OIDC when a human user's identity is needed.

Security implications	API keys: long-lived, easy to leak, no user identity. JWT: stateless but hard to revoke; keep lifetime short and validate signature, issuer, audience and expiry. Sessions: need CSRF protection. Refresh tokens: store encrypted, rotate on use.
Lab	Add JWT login and role checks to the Client Registry API; write tests for 401, 403 and cross-user access (API1).
Official source	OWASP API Security Top 10; OAuth 2.0 (RFC 6749); JWT (RFC 7519)
Assessment	OWASP oral check; security tests green

Level 6: Backend Frameworks

Topics	**FastAPI** (primary) · Django REST Framework · Node.js · Express.js · NestJS · Spring Boot only where a client's stack requires it
Business purpose	Choose the framework the client's team can maintain, not the one you like.
Selection guide	FastAPI: new Python APIs and integration services, automatic OpenAPI. Django REST: admin UI and ORM-heavy back-office. Express/NestJS: client has a TypeScript team. Spring Boot: client mandates Java.
Lab	Rebuild the same two endpoints in FastAPI and NestJS; compare code size, validation and docs.
Official source	fastapi.tiangolo.com · django-rest-framework.org · nodejs.org/docs

Level 7: Databases

Topics	PostgreSQL · MySQL · MongoDB where justified · SQL · relationships · transactions · indexes · migrations · connection pooling · consistency · backup and recovery
Business purpose	Integrations need durable state: what was synced, what failed, what was already processed.
Lab	Model clients, contacts and a sync-log table; write a migration; add a unique index for idempotency; back up and restore.
Common errors	Missing index on lookup column · long transactions · forgetting to test restore · storing secrets in tables
Official source	postgresql.org/docs

Level 8: Integration Architecture

Topics	Point-to-point · middleware · API gateway · webhooks · message queues · event-driven architecture · brokers · retry · idempotency · dead-letter queues · eventual consistency · distributed transactions and their limits
Business purpose	This is what makes integrations survive real life.
Key ideas	**At-least-once delivery** means duplicates will happen, so consumers must be idempotent. **Dead-letter queue**: messages that exhaust retries are parked for human review, not dropped. **Eventual consistency**: both systems agree *after a short delay*; agree the delay with the client. **Distributed transactions** (two-phase commit across vendors) are rarely possible; use sagas with compensating actions instead.

Tested reference code (run it with `python retry_idempotent.py`; tests in `snippets/`):

```python
"""Retry with exponential back-off + jitter, and an idempotent consumer."""
import random
import time

RETRYABLE = {429, 500, 502, 503, 504}


def call_with_retry(send, max_attempts=5, base=0.5, cap=30.0, sleep=time.sleep):
    """`send()` returns (status_code, body, retry_after_seconds_or_None)."""
    for attempt in range(1, max_attempts + 1):
        status, body, retry_after = send()
        if status < 400:
            return body
        if status not in RETRYABLE:                    # 4xx: do NOT retry blindly
            raise ValueError(f"non-retryable status {status}")
        if attempt == max_attempts:
            raise TimeoutError(f"gave up after {attempt} attempts (last {status})")
        delay = retry_after if retry_after else min(cap, base * 2 ** (attempt - 1))
        sleep(delay + random.uniform(0, delay * 0.25))   # jitter avoids thundering herd


class IdempotentConsumer:
    """Process each event id once, even if the broker delivers it many times."""

    def __init__(self):
        self.seen: set[str] = set()      # production: a unique-key table in PostgreSQL
        self.created: list[dict] = []

    def handle(self, event: dict) -> str:
        key = event["event_id"]
        if key in self.seen:
            return "duplicate-ignored"
        self.created.append({"invoice_for": event["order_id"]})
        self.seen.add(key)               # production: insert key + effect in ONE transaction
        return "processed"


if __name__ == "__main__":
    calls = iter([(503, None, None), (429, None, 1), (200, {"ok": True}, None)])
    print(call_with_retry(lambda: next(calls), sleep=lambda s: print(f"  wait {s:.2f}s")))
    c = IdempotentConsumer()
    evt = {"event_id": "evt_1", "order_id": "A-100"}
    print([c.handle(evt) for _ in range(3)], len(c.created))
```

Lab	Projects 5, 6 and 7
Official source	rabbitmq.com/docs · kafka.apache.org/documentation
Assessment	Failure-scenario test report

Level 9: API Documentation and Testing

Topics	OpenAPI · Swagger · Postman · automated API tests · contract tests · integration tests · load tests · mock servers · versioning
Business purpose	Documentation and tests are what the client pays to keep; they also let the next engineer work safely.
Lab	Export OpenAPI from your service; run it as a contract in CI; mock a vendor API; run a load test and record p95 latency.
Official source	openapis.org · learning.postman.com · opentelemetry.io/docs

7.10 The 16-point teaching format

Every lesson and assignment in this programme contains all sixteen parts. Trainers must not drop any.

#	Part	#	Part
1	Definition	9	Complete code examples
2	Business purpose	10	Expected output
3	Real-world example	11	Verification
4	Prerequisites	12	Common errors
5	Required tools	13	Security requirements
6	Installation	14	Documentation
7	Configuration	15	Practical assignment
8	Step-by-step implementation	16	Assessment

7.11 Technology stack (what we standardise on)

Category	Tools
IDE	VS Code
Languages	Python (primary), TypeScript
Frameworks	FastAPI, Django, NestJS

Category	Tools
Database	PostgreSQL, MySQL
API testing	Postman, Insomnia
Documentation	OpenAPI, Swagger
Version control	Git, GitHub
Containers	Docker
Messaging	RabbitMQ, Kafka
Automation	n8n, Make where suitable
API gateway	Kong, cloud-native gateways
Monitoring	Prometheus, Grafana
Logging	OpenTelemetry, a suitable log platform
Cloud	AWS, Azure, GCP
Deployment	Docker, CI/CD

Start with **one** primary backend language. Introduce others only when project requirements justify them.

VEL SOFTWARE SOLUTIONS

Part 8: First Practical Assignment: The Client Registry API

Project 1 of 10 · Week 4 · Gate 1 deliverable. The complete reference solution is in `assignment-01/` and its 12 automated tests pass.

8.1 Definition

An **API** (Application Programming Interface) is a contract that lets one program ask another program to do something or return data, using agreed requests and responses. A **REST API** does this over HTTP using resources (nouns such as `clients`) and standard methods (`GET`, `POST`, `PATCH`, `DELETE`).

8.2 Business purpose

VEL needs a **Client Registry**: the first internal API in the department's inventory. Later projects connect it to PostgreSQL, add OAuth, and sync it with a CRM. Building it teaches the habits that every client API needs: validation, a standard error shape, authentication, pagination and tests.

8.3 Real-world example

A sales app sends `POST /v1/clients` when a lead is won. The accounting app later calls `GET /v1/clients?status=active` to create customer records. Neither needs to know how the other stores data; they share only the API contract.

8.4 Prerequisites

Week 1 to 3 complete · comfortable with Python functions, dicts and exceptions · know HTTP methods and status codes · Git basics.

8.5 Required tools

Python 3.12+ · VS Code · Git · Postman or Insomnia · a terminal.

8.6 Installation

```
cd assignment-01
python -m venv .venv
source .venv/bin/activate          # Windows PowerShell: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

8.7 Configuration

Copy `.env.example` to `.env`, set a long random key, and **never commit** `.env` (it is in `.gitignore`).

```
cp .env.example .env
# generate a key:
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

`.env.example`:

```
# Copy to .env and fill in. NEVER commit .env.
CLIENT_API_KEY=change-me-to-a-long-random-value
```

`requirements.txt`:

```
fastapi>=0.110,<1.0
uvicorn[standard]>=0.29
pydantic>=2.6
pytest>=8.0
httpx>=0.27
```

8.8 Step-by-step implementation

Specification you must meet

Method and path	Purpose	Success	Errors
`GET /health`	Liveness (public)	200	none
`POST /v1/clients`	Create client	201 + `Location` header	401, 409 duplicate email, 422
`GET /v1/clients`	List with `page`, `page_size`, `status`, `industry`, `sort`	200	401, 422
`GET /v1/clients/{id}`	Read one	200	401, 404
`PATCH /v1/clients/{id}`	Partial update	200	401, 404, 409, 422
`DELETE /v1/clients/{id}`	Delete	204	401, 404

Order of work (do each step, then commit):

1. Create the folder layout: `app/` (code) and `tests/`.
2. **Schemas first** (`schemas.py`): the contract. Validation lives here.
3. **Store** (`store.py`): an in-memory class; no HTTP knowledge.
4. **Errors** (`errors.py`): one error shape for every failure.
5. **Security** (`security.py`): API-key dependency using constant-time comparison.
6. **Routes** (`main.py`): thin handlers, request-ID middleware, logging.
7. **Tests** (`tests/test_clients.py`): write the failing cases first, then make them pass.
8. Run the server, explore `/docs`, and call it from Postman.
9. Write the README and export the OpenAPI file.

8.9 Complete code

app/schemas.py : request and response models

```
"""Request and response models (the API contract)."""
from datetime import datetime
from typing import Literal

from pydantic import BaseModel, Field

Status = Literal["lead", "active", "inactive"]


class ClientCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    email: str = Field(pattern=r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
    industry: str = Field(min_length=2, max_length=60)
    status: Status = "lead"


class ClientUpdate(BaseModel):
    """PATCH body: every field optional."""
    name: str | None = Field(default=None, min_length=2, max_length=100)
    email: str | None = Field(default=None, pattern=r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
    industry: str | None = Field(default=None, min_length=2, max_length=60)
    status: Status | None = None


class Client(ClientCreate):
    id: int
    created_at: datetime


class Page(BaseModel):
    items: list[Client]
    page: int
    page_size: int
    total: int
```

app/store.py : storage (replaced by PostgreSQL in Project 2)

```
"""In-memory store. Assignment 02 replaces this with PostgreSQL."""
from datetime import datetime, timezone

from .schemas import Client, ClientCreate, ClientUpdate


class ClientStore:
    def __init__(self) -> None:
        self._rows: dict[int, Client] = {}
        self._next_id = 1

    def create(self, data: ClientCreate) -> Client:
        row = Client(id=self._next_id, created_at=datetime.now(timezone.utc), **data.model_dump())
        self._rows[row.id] = row
        self._next_id += 1
        return row

    def get(self, client_id: int) -> Client | None:
        return self._rows.get(client_id)

    def email_exists(self, email: str, exclude_id: int | None = None) -> bool:
        return any(r.email.lower() == email.lower() and r.id != exclude_id for r in
self._rows.values())

    def update(self, client_id: int, data: ClientUpdate) -> Client | None:
        row = self._rows.get(client_id)
```

```
        if row is None:
            return None
        updated = row.model_copy(update=data.model_dump(exclude_unset=True))
        self._rows[client_id] = updated
        return updated

    def delete(self, client_id: int) -> bool:
        return self._rows.pop(client_id, None) is not None

    def all(self) -> list[Client]:
        return list(self._rows.values())
```

app/errors.py : the company error standard

```
"""One error shape for the whole API (company standard)."""
from fastapi import FastAPI, HTTPException, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse


class ApiError(Exception):
    def __init__(self, status: int, code: str, message: str) -> None:
        self.status, self.code, self.message = status, code, message


def _body(request: Request, code: str, message: str, details: object = None) -> dict:
    return {"error": {"code": code, "message": message, "details": details,
                      "request_id": getattr(request.state, "request_id", None)}}


def install_error_handlers(app: FastAPI) -> None:
    @app.exception_handler(ApiError)
    async def _api(request: Request, exc: ApiError) -> JSONResponse:
        return JSONResponse(_body(request, exc.code, exc.message), status_code=exc.status)

    @app.exception_handler(RequestValidationError)
    async def _validation(request: Request, exc: RequestValidationError) -> JSONResponse:
        details = [{"field": ".".join(str(p) for p in e["loc"][1:]), "problem": e["msg"]} for e in
exc.errors()]
        return JSONResponse(_body(request, "validation_error", "Request validation failed",
details), status_code=422)

    @app.exception_handler(HTTPException)
    async def _http(request: Request, exc: HTTPException) -> JSONResponse:
        return JSONResponse(_body(request, "http_error", str(exc.detail)),
status_code=exc.status_code)
```

app/security.py : API-key authentication

```
"""API-key authentication. Key comes from the environment, never from code."""
import hmac
import os

from fastapi import Header

from .errors import ApiError


def require_api_key(x_api_key: str | None = Header(default=None)) -> None:
    expected = os.environ.get("CLIENT_API_KEY", "")
    if not expected:
        raise ApiError(500, "server_misconfigured", "Server API key is not configured")
    if not x_api_key or not hmac.compare_digest(x_api_key, expected):
        raise ApiError(401, "unauthorized", "Missing or invalid API key")
```

app/main.py : routes, middleware, logging

```python
"""Client Registry API v1 - VEL Assignment 01."""
import logging
import uuid
from typing import Literal

from fastapi import Depends, FastAPI, Query, Request, Response

from .errors import ApiError, install_error_handlers
from .schemas import Client, ClientCreate, ClientUpdate, Page, Status
from .security import require_api_key
from .store import ClientStore

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("client-registry")

app = FastAPI(title="Client Registry API", version="1.0.0")
install_error_handlers(app)
store = ClientStore()


@app.middleware("http")
async def request_id_and_logging(request: Request, call_next):
    request.state.request_id = request.headers.get("x-request-id") or str(uuid.uuid4())
    response = await call_next(request)
    response.headers["X-Request-ID"] = request.state.request_id
    log.info("%s %s -> %s [%s]", request.method, request.url.path, response.status_code,
request.state.request_id)
    return response


@app.get("/health")
def health() -> dict:
    return {"status": "ok"}


v1 = Depends(require_api_key)


@app.post("/v1/clients", response_model=Client, status_code=201, dependencies=[v1])
def create_client(body: ClientCreate, response: Response) -> Client:
    if store.email_exists(body.email):
        raise ApiError(409, "duplicate_email", "A client with this email already exists")
    row = store.create(body)
    response.headers["Location"] = f"/v1/clients/{row.id}"
    return row


@app.get("/v1/clients", response_model=Page, dependencies=[v1])
def list_clients(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    status: Status | None = None,
    industry: str | None = None,
    sort: Literal["name", "-name", "created_at", "-created_at"] = "name",
) -> Page:
    rows = store.all()
    if status:
        rows = [r for r in rows if r.status == status]
    if industry:
        rows = [r for r in rows if r.industry.lower() == industry.lower()]
    field = sort.lstrip("-")
    rows.sort(key=lambda r: getattr(r, field), reverse=sort.startswith("-"))
    start = (page - 1) * page_size
    return Page(items=rows[start:start + page_size], page=page, page_size=page_size,
total=len(rows))


@app.get("/v1/clients/{client_id}", response_model=Client, dependencies=[v1])
def get_client(client_id: int) -> Client:
    row = store.get(client_id)
```

```
    if row is None:
        raise ApiError(404, "not_found", f"Client {client_id} not found")
    return row


@app.patch("/v1/clients/{client_id}", response_model=Client, dependencies=[v1])
def update_client(client_id: int, body: ClientUpdate) -> Client:
    if body.email and store.email_exists(body.email, exclude_id=client_id):
        raise ApiError(409, "duplicate_email", "A client with this email already exists")
    row = store.update(client_id, body)
    if row is None:
        raise ApiError(404, "not_found", f"Client {client_id} not found")
    return row


@app.delete("/v1/clients/{client_id}", status_code=204, dependencies=[v1])
def delete_client(client_id: int) -> Response:
    if not store.delete(client_id):
        raise ApiError(404, "not_found", f"Client {client_id} not found")
    return Response(status_code=204)
```

`tests/test_clients.py` : automated tests (success **and** failure)

```
import pytest
from fastapi.testclient import TestClient

from app import main

KEY = {"X-API-Key": "test-key"}
GOOD = {"name": "Acme Traders", "email": "ops@acme.example", "industry": "retail"}


@pytest.fixture(autouse=True)
def fresh(monkeypatch):
    monkeypatch.setenv("CLIENT_API_KEY", "test-key")
    main.store.__init__()          # reset in-memory data between tests
    return TestClient(main.app)


@pytest.fixture
def c():
    return TestClient(main.app)


def test_health_is_public(c):
    assert c.get("/health").json() == {"status": "ok"}


def test_create_returns_201_and_location(c):
    r = c.post("/v1/clients", json=GOOD, headers=KEY)
    assert r.status_code == 201
    assert r.headers["Location"] == "/v1/clients/1"
    assert r.json()["status"] == "lead"


def test_missing_key_is_401_in_standard_shape(c):
    r = c.get("/v1/clients")
    assert r.status_code == 401
    assert r.json()["error"]["code"] == "unauthorized"


def test_wrong_key_is_401(c):
    assert c.get("/v1/clients", headers={"X-API-Key": "nope"}).status_code == 401


def test_validation_error_lists_fields(c):
    r = c.post("/v1/clients", json={"name": "A", "email": "bad"}, headers=KEY)
    assert r.status_code == 422
    fields = {d["field"] for d in r.json()["error"]["details"]}
```

```
    assert {"name", "email", "industry"} <= fields


def test_duplicate_email_is_409(c):
    c.post("/v1/clients", json=GOOD, headers=KEY)
    r = c.post("/v1/clients", json={**GOOD, "name": "Other"}, headers=KEY)
    assert r.status_code == 409


def test_get_unknown_is_404(c):
    assert c.get("/v1/clients/99", headers=KEY).status_code == 404


def test_patch_changes_only_sent_fields(c):
    c.post("/v1/clients", json=GOOD, headers=KEY)
    r = c.patch("/v1/clients/1", json={"status": "active"}, headers=KEY)
    assert r.json()["status"] == "active" and r.json()["name"] == "Acme Traders"


def test_delete_then_404(c):
    c.post("/v1/clients", json=GOOD, headers=KEY)
    assert c.delete("/v1/clients/1", headers=KEY).status_code == 204
    assert c.get("/v1/clients/1", headers=KEY).status_code == 404


def test_pagination_filter_sort(c):
    for i, (n, s) in enumerate([("Zeta Ltd", "active"), ("Alpha Ltd", "lead"), ("Mid Ltd",
"active")]):
        c.post("/v1/clients", json={**GOOD, "name": n, "email": f"x{i}@e.example", "status": s},
headers=KEY)
    r = c.get("/v1/clients?status=active&sort=name&page_size=1&page=2", headers=KEY).json()
    assert r["total"] == 2 and [x["name"] for x in r["items"]] == ["Zeta Ltd"]


def test_bad_page_size_rejected(c):
    assert c.get("/v1/clients?page_size=1000", headers=KEY).status_code == 422


def test_request_id_echoed(c):
    r = c.get("/health", headers={"X-Request-ID": "abc-123"})
    assert r.headers["X-Request-ID"] == "abc-123"
```

8.10 Expected output

Run the tests:

```
python -m pytest -q
```

```
............                                                             [100%]
12 passed
```

Start the server and call it:

```
export CLIENT_API_KEY="$(python -c 'import secrets; print(secrets.token_urlsafe(32))')"
uvicorn app.main:app --reload
```

```
curl -s -X POST localhost:8000/v1/clients \
  -H "X-API-Key: $CLIENT_API_KEY" -H "Content-Type: application/json" \
  -d '{"name":"Acme Traders","email":"ops@acme.example","industry":"retail"}'
```

{"name":"Acme Traders","email":"ops@acme.example","industry":"retail","status":"lead","id":1,"created_at":"2026-11-04T09:30:00.123456Z"}

Missing key gives `401` with `{"error":{"code":"unauthorized",...}}`. Interactive docs are at `http://localhost:8000/docs`.

8.11 Verification checklist

#	Check	Done
1	All tests pass, including failure cases	☐
2	Every error response uses the standard shape and includes `request_id`	☐
3	Request without `X-API-Key` is rejected on every `/v1` route	☐
4	`page_size=1000` is rejected (resource-consumption control)	☐
5	Duplicate email returns 409	☐
6	Logs show method, path, status and request ID, and **no key**	☐
7	`git log` shows small commits; `git status` shows no `.env`	☐
8	A teammate can run it from your README alone	☐

8.12 Common errors

Symptom	Cause	Fix
`ModuleNotFoundError: app`	Running pytest from the wrong folder	Run from `assignment-01/`
`500 server_misconfigured`	`CLIENT_API_KEY` not set	Export it or load `.env`
`422` on a valid-looking body	Missing `Content-Type: application/json` or wrong field names	Check the details array
Tests leak data between cases	Store not reset	The fixture reinitialises the store
`401` in Postman	Header name typo	Use exactly `X-API-Key`

8.13 Security requirements

- Key from the environment only; compared in constant time; never logged.
- Validation rejects unknown shapes before any logic runs.
- Page size capped (API4). Errors never expose internals (API8).
- No real client data in tests or in the repository.

- Known limitation to report in your README: one shared key means **no object-level authorisation yet**. That is solved in Project 3.

8.14 Documentation to submit

`README.md` (purpose, setup, run, test, example calls) · exported `openapi.json` / `openapi.yaml` · Postman collection (no secrets; use variables) · a short **error catalogue** listing each `code`.

8.15 Practical assignment (what you must deliver)

1. Re-create the API **yourself** from the specification (do not copy-paste; reading the reference is allowed once you are stuck for 30 minutes).
2. **Extend it** with: (a) `contacts` as a sub-resource of a client (`/v1/clients/{id}/contacts`); (b) a `search` query parameter on the list endpoint; (c) tests for each.
3. Add a `/v1/clients/export` endpoint design note in the README: how would you stream 1 million rows safely?
4. Open a pull request using the template and respond to review comments.

8.16 Assessment

Marked with the rubric in section 6.4. **Must-have for Gate 1:** all tests green · standard errors · authentication · pagination cap · no secrets in Git · README. **Stretch:** OpenAPI examples, structured JSON logging, request-size limit.

8.17 Module release plan (what comes next)

Module	Content	Released when
2	Levels 1 to 3 deep lessons with full lab code	Start of week 1
3	Level 4 REST design workshop and Postman	Week 3
4	**Project 2:** PostgreSQL, migrations, pooling	After Gate 1
5	**Project 3:** OAuth 2.0, JWT, RBAC	Week 7
6	**Project 4:** two-app integration	Week 8
7	Projects 5 to 7: webhooks, sync, retry, idempotency	Week 10
8	Projects 8 and 9: monitoring and Docker	Week 12
9	Capstone brief and panel guide	Week 13

Part 9: Internal Portal, Commercial Pricing and Client Acquisition

9.1 Internal management portal: modules

Module	Purpose	Key fields
Client management	Accounts, contacts, contracts	Industry, owner, SLA tier
API inventory	Section 4.13 register	Version, consumers, lifecycle
Integration inventory	Each connector and its systems	Source, destination, frequency
Project tracking	Stage and gate status	Stage, gate, risks
Employee management	Roles, grade, training level	Gate status, mentor
Task assignment	Tickets and time	Assignee, estimate, actual
API usage tracking	Calls per client/API	Volume, errors
Error monitoring	Failed syncs and exceptions	Count, age, owner
Incident tracking	Sev levels and reports	Timeline, root cause
Cost tracking	Infrastructure and vendor costs	Per client / integration
Billing	Invoices, usage charges	Plan, usage, status
Documentation	Links to handover packs	Version, owner
Support tickets	Client requests	Priority, SLA clock
SLA tracking	Response and resolution	Breach alerts

> **The portal never stores secrets.** It stores *references* (for example "stored in vault path X, owner Y, rotates 2027-01-15"), never the secret itself.

9.2 Pricing components

API development fee · integration setup fee · per-system integration fee · monthly monitoring fee · maintenance fee · API usage charges · cloud hosting (passed through or bundled, stated clearly) · third-party vendor expenses (passed through at cost, with the vendor named) · support charges. Add applicable taxes (for example GST in India) as required by current law; confirm rates with your accountant.

9.3 What to calculate

Measure	Formula
Development labour	Σ(hours × cost rate)
Infrastructure cost	Hosting + database + broker + monitoring per month
Third-party cost	Vendor subscriptions + per-call fees
Cost per integration	(One-time costs ÷ planned months of life) + monthly recurring costs
Cost per transaction	Total monthly cost ÷ monthly transactions
Cost per customer	Total monthly cost for the client
Gross margin	(Revenue − direct cost) ÷ Revenue
Contribution margin	(Revenue − variable cost) ÷ Revenue
Break-even	Fixed monthly company cost ÷ average monthly contribution per client
Recurring revenue	Monitoring + maintenance + base usage fees per month

9.4 Usage-based pricing model

Use a **base fee including an allowance**, then a **per-transaction charge** above it, so revenue rises when vendor and compute costs rise.

Worked example (illustrative numbers only: replace with verified costs, Stage 06):

Item	Value
Monthly fixed cost for this client (hosting, monitoring, share of support)	₹10,000
Variable cost per transaction (compute, queue, vendor call, logs)	₹0.40
Monthly transactions	50,000
Base fee (includes 20,000 transactions)	₹25,000
Overage price	₹1.20 per transaction

Result	Calculation	Value
Variable cost	50,000 × 0.40	₹20,000
Total cost	10,000 + 20,000	₹30,000
Cost per transaction	30,000 ÷ 50,000	₹0.60
Revenue	25,000 + (30,000 × 1.20)	₹61,000
Gross margin	(61,000 − 30,000) ÷ 61,000	50.8%

Result	Calculation	Value
Contribution margin	(61,000 − 20,000) ÷ 61,000	67.2%
Break-even (illustrative company overhead ₹3,00,000 per month)	3,00,000 ÷ 41,000 contribution per client	7.3, so **8 such clients**

9.5 Target clients

SaaS businesses · e-commerce · healthcare · manufacturing · logistics · financial services · ERP and CRM users · any business running multiple disconnected applications. **Regulated sectors** (healthcare, finance) need extra compliance review at Stage 02; do not accept those projects without Security and Founder approval.

Offer statement: "We connect your existing business applications so your teams can exchange data automatically, reduce manual entry and maintain consistent records."

9.6 Sales document set

Document	Stage	Owner
Discovery questionnaire	02	Architect
API feasibility report	04	Sr. Engineer
Integration proposal	05 to 06	PM
Architecture diagram	05	Architect
Commercial quotation	06	PM, approved by Founder
Statement of Work (SOW)	06	PM / Founder
Demo	12 to 13	Engineer
Test report	12	QA
Handover checklist	15	PM

9.7 The ten practical projects

#	Project	Skills proven	Week
1	Basic REST API	REST, validation, errors, tests	4
2	API connected to PostgreSQL	SQL, migrations, pooling	6
3	API authentication	API keys, OAuth/JWT, RBAC	7
4	Integrate two sample applications	Mapping, transformation, sync	8 to 9
5	Webhook receiver	Signatures, fast ack, replay defence	10

#	Project	Skills proven	Week
6	Scheduled synchronisation service	Cursors, pagination, scheduling	11
7	Retry and idempotency	Back-off, dedupe, dead-letter	11
8	API monitoring dashboard	Prometheus, Grafana, alerts	12
9	Containerise the integration	Docker, compose, health checks	12
10	Full simulated enterprise integration	Stages 03 to 15 end-to-end	13

9.8 Official learning sources

Topic	Source
Web fundamentals	MDN Web Docs: developer.mozilla.org
OpenAPI	openapis.org
FastAPI	fastapi.tiangolo.com
Django REST Framework	django-rest-framework.org
Node.js	nodejs.org/docs
PostgreSQL	postgresql.org/docs
Postman	learning.postman.com
Docker	docs.docker.com
OWASP API Security	owasp.org/www-project-api-security
RabbitMQ	rabbitmq.com/docs
Apache Kafka	kafka.apache.org/documentation
OpenTelemetry	opentelemetry.io/docs

Appendices: Templates and Reference

Appendix A: Lead qualification report

Field	Entry
Client / industry	
Contact and decision-maker	
Systems to connect	
Business problem and cost today	
Applications (count)	
Data volume (per day / peak)	
Required automation	
Budget range / approver	
Timeline and reason	
Score (Red / Amber / Green): problem · decision-maker · API likelihood · budget · timeline	
Recommendation	Go to discovery / Decline / Nurture

Appendix B: Discovery questionnaire

1. Which applications must exchange data? Name, edition, version, hosting.
2. Does each have a documented API? Which plan or licence includes it? Who is the vendor contact?
3. Which data entities move, and in which direction? Which system is the system of record?
4. How is data moved today? By whom, how often, with what error rate?
5. What triggers a transfer: event, schedule, manual approval?
6. How many records per day, per peak hour? Growth expected in 12 months?
7. Authentication each system supports. Who can issue credentials? Is there a sandbox?
8. Known rate limits, maintenance windows, data-size limits.
9. What must happen if the destination is down for 1 hour? 1 day?
10. Which data is personal, financial or health data? Residency and retention rules?
11. Who approves production access? Who will support after go-live?
12. What does success look like, in numbers?

Appendix C: Integration specification template

Section	To complete
1. Purpose and scope	

Section	To complete
2. Systems, versions, environments	
3. Entities and system of record	
4. Field mapping table (source, destination, type, required, transformation, example)	
5. Triggers and frequency	
6. Authentication per system	
7. Error handling and retry policy	
8. Idempotency key	
9. Logging and monitoring	
10. Non-functional: volume, latency, availability	
11. Acceptance criteria (measurable)	
12. Assumptions, risks, out of scope	
13. Change log and signatures	

Appendix D: Weekly report (every Friday)

Done this week	Learned	Blocked / needs help	Next week	Hours	Risks

Appendix E: Pull request checklist

☐ Linked ticket · ☐ Tests added and passing · ☐ No secrets or client data · ☐ Docs / OpenAPI updated · ☐ Errors use the standard shape · ☐ Logging contains no sensitive data · ☐ Migration is reversible · ☐ Reviewer assigned

Appendix F: Incident report template

Field	Entry
Incident ID, severity (Sev-1 critical · Sev-2 major · Sev-3 minor)	
Detected (time, how)	
Impact (clients, records, duration)	
Timeline	
Root cause	
Fix and recovery steps	
Data affected and how it was reconciled	

Field	Entry
Prevention actions (owner, date)	
Client communication sent	

Appendix G: Client handover checklist

☐ API documentation · ☐ Integration architecture · ☐ Data-mapping document · ☐ Authentication guide · ☐ Deployment guide · ☐ Error handling guide · ☐ Monitoring guide · ☐ Troubleshooting guide · ☐ Test report · ☐ Access register (no secrets) · ☐ Maintenance agreement · ☐ Credential rotation dates · ☐ Training session held · ☐ Signed acceptance

Appendix H: Glossary

Term	Meaning
API	Defined way for programs to talk to each other
REST	Style of API using HTTP methods on resources
Webhook	A push notification: the vendor calls *your* URL when an event happens
Idempotent	Repeating the operation has the same effect as doing it once
Dead-letter queue	Holding area for messages that failed all retries
Eventual consistency	Systems agree after a short delay, not instantly
Rate limit	Maximum calls allowed in a time window
OAuth 2.0	Standard for delegated, scoped access with tokens
JWT	Signed token carrying claims, with an expiry
RBAC	Access by role
OpenAPI	Machine-readable description of an API
SLA	Agreed response and resolution times
Sandbox	Vendor test environment with no real data

Appendix I: Coverage map: your master prompt vs this volume

Master-prompt section	Where covered
1 Role	Parts 0 to 2
2 Service portfolio	1.2
3 Training roadmap (Levels 1 to 9)	Part 7, Part 6
4 Technology stack	7.11, Part 1.4

Master-prompt section	Where covered
5 Stages 01 to 16	Part 3
6 Governance	Part 4
7 Internal portal	9.1
8 Commercial pricing	9.2 to 9.4
9 Client acquisition	9.5, 9.6
10 Ten projects	9.7, 6.2
11 Learning sources	9.8
12 Teaching format	7.10, applied in Part 8
13 First action: handbook, workflow, onboarding, 90-day plan, responsibilities, first assignment	Parts 0 to 6 and Part 8

Appendix J: Known limits of this volume

- Price figures are **illustrative**. Verify vendor prices from official pricing pages and record the date.
- Legal, tax and data-protection requirements differ by client and country. Confirm with qualified advisers before signing contracts.
- Vendor APIs change. Re-verify documentation at Stage 04 and during maintenance.
- Full lesson code for Levels 1 to 3 and Projects 2 to 10 is released in later modules (8.17).

End of Module 1.